Advanced Algorithms: Lecture 8 & 9 Notes

Dana Randall Spring 2024

Sarthak Mohanty

Online Algorithms

Before this class, our algorithm design has been as follows: take in some input, put it through some black box, and produce some output.

In the last section, we expanded our definition a little, by allowing some randomized "bits" to be fed in alongside our input. We saw that this approach allowed for algorithms that came with security and time complexity benefits.

Now, instead of allowing our algorithm more information, we *restrict* some information to our algorithm.

Definition. An **online algorithm** is one that produces its output step-by-step, as more parts of the input become available. More formally, the input of an online algorithm is a sequence of requests $\sigma = \sigma_1, \ldots, \sigma_n$. The algorithm needs to handle these requests in order. When handling σ_t, the algorithm does not know the future requests $\sigma_{t'}$ for $t' > t$.

While you may not have encountered online algorithms formally yet, there are many areas you have seen similar motifs. For example, online algorithms and machine learning are both concerned with problems of making decisions from limited information. In fact, there are a collection of results in Computational Learning Theory that fit nicely into the "online algorithms" framework, such as the Winnow algorithm. [1]

Paging

Computer systems often have multi-level storage systems. Earlier, there were only RAM and long-term storage devices. Nowadays there are also several levels of cache memory (L1, L2, L3 cache).

[1] Professor Vempala, who taught this class last semester, covered this here.

The setting for this problem is as follows: we have a two-level computer memory system, composed of fast memory (of size k), and slow memory (of size $n > k$). We need to answer a request sequence $\sigma = \sigma_1\sigma_2 \dots \sigma_m$, where σ_i is an item to be paged.

Any page someone wants needs to be moved to fast memory. If fast memory (cache) is full and I need a page that is not in it, we encounter a page fault where I now need to swap in a page.

Offline Approach

Suppose I already have a list of all the pages I intend on accessing (i.e. P5, P7, P5, P1, P1000, etc.). Because we know the order of pages to be accessed, we can deterministically find an optimal strategy to minimize page faults. One such strategy is **longest forward distance** (LFD), where we replace the page whose next request is latest in the future.

Theorem. LFD is an optimal page replacement policy.

Proof. Fix a sequence σ. For all i, let $C_0^{(i)}$ denote the state of the cache after request σ_i is served, and let $C_0 = (C_0^{(i)})_{1 \le i \le m}$.

Let $C_t = (C_t^{(i)})_i$ be defined inductively as follows: up to and including time t, use LFD to choose which page to evict when there is a page fault. This determines the state of the cache at each timestep until time t. Then, follow the choice of evictions of C_{t-1} for all times $> t$, except for one modification defined as follows.

Assume that at time t, algorithm C_{t-1} evicts page u, whereas LFD evicts page v. Let t' be the first time when either C_{t-1} evicts page v, or C_{t-1} brings page u back into the cache by evicting some page u', whichever comes first. In the first case, at time t', C_t evicts page u. In the second case, at time t', C_t evicts page u'. Either way, after serving request t', the state of the cache is the same for C_{t-1} and C_t.

By definition of LFD, between time t and time t', there can be no request for page v, so C_t is well-defined and incurs no more page faults than C_{t-1}. By induction, C_t is at least as good as C_0. For $t = m$, C_m is exactly LFD, so LFD is at least as good as C_0. This was for an arbitrary C_0, so LFD is optimal.

Online Approach

Definition: Let $OPT(\sigma)$ and $ALG(\sigma)$ denote the cost incurred by the best possible offline algorithm OPT, and the online algorithm ALG, respectively. The **competitive ratio** of a deterministic online algorithm ALG is defined as

$$\max_{\sigma} \frac{ALG(\sigma)}{OPT(\sigma)}.$$

Sometimes we choose to ignore constant additive factors in the cost of the algorithm. This leads to the following alternative definition of competitive ratio. In this case, when the algorithm's cost involves no constant additive factor, we call the corresponding ratio a strong competitive ratio.

Definition: A deterministic online algorithm is **c-competitive** iff

$$\exists a \geq 0 \text{ s.t. } \forall \sigma, ALG(\sigma) \leq c \cdot OPT(\sigma) + a.$$

We first state a lower bound for deterministic online algorithms:

Theorem. For any deterministic online paging algorithm A, the competitive ratio of A is at least k.

Proof. Consider a universe of $k+1$ pages. Since A has only k pages in cache, there is always at least one page that is not in A's cache. An adversary constructs the request sequence by always requesting such a page. Hence A faults on every request, so

$$A(\sigma) = |\sigma|.$$

On the other hand, OPT can serve this sequence with at most one fault per k requests, up to an additive constant. Equivalently,

$$OPT(\sigma) \leq \frac{|\sigma|}{k} + O(1).$$

Therefore,

$$\frac{A(\sigma)}{OPT(\sigma)} \geq \frac{|\sigma|}{|\sigma|/k + O(1)}.$$

As $|\sigma| \to \infty$, this approaches k. Thus no deterministic online algorithm has competitive ratio better than k.

From the above result, we can categorize common paging strategies by whether they are k-competitive or not. Some k-competitive strategies include:

Least Recently Used (LRU)
Replace the page that has not been used for the longest time.

First in First Out (FIFO)
Replace the page that has been in fast memory longest.

Strategies that do not achieve this competitive bound include:

Last In First Out (LIFO)
Replace the page most recently moved to fast memory.

Least Frequently Used (LFU)
Replace the page that has been used the least.

Marking Algorithms

Marking algorithms are a class of online paging algorithms that achieve the k-competitive bound. In fact, all the above-mentioned competitive algorithms are some form of a marking algorithm.

A marking algorithm has a design which can be characterized as follows:

- Partition the sequence of accesses σ into phases, where each phase contains k *distinct* pages, and a new phase begins when a request arrives for the $(k+1)$st distinct page since the start of the current phase.
- Mark each new page requested during the current phase.
- At the end of a phase, unmark all pages in fast memory.

Note that a marking algorithm never evicts a page which is already marked. This key observation leads to an easy analysis of these algorithms:

Theorem. All deterministic marking algorithms are k-competitive.

Proof. Partition the request sequence into phases, where each phase contains k distinct pages, and a new phase begins when a request arrives for the $(k+1)$st distinct page since the start of the current phase.

A marking algorithm faults at most k times per phase. This is because once a page is requested in a phase, it is marked, and marked pages are never evicted. Therefore each distinct page in a phase can cause at most one fault.

Now consider OPT. Let p be the first request of a phase after the first phase.

The previous phase contained k distinct pages, none of which is p; otherwise p would not start a new phase. Thus, across the previous phase and the first request of the current phase, there are $k+1$ distinct pages. Since OPT has only k cache slots, it must fault at least once.

Therefore, if there are q phases, then

$$A(\sigma) \leq kq$$

and

$$OPT(\sigma) \geq q - O(1).$$

Hence

$$A(\sigma) \leq k \cdot OPT(\sigma) + O(k).$$

Thus every deterministic marking algorithm is k-competitive.

Randomization

The competitive ratio that we have achieved thus far is pretty substandard. However, it works rather well in practice. This is because competitive ratio measures the "worst-case" performance, but the distribution of the input sequence usually makes it so our average performance is pretty good.

The issue remains that if an adversary wanted to defeat our system, it would be pretty easy. We know already that one way to beat this adversary is to randomize our decision process. Of course, this requires the adversary to produce the sequence in advance – after knowing our algorithm, but before the algorithm responds to the sequence. This is called an *oblivious adversary*.

For example, consider this randomized marking algorithm, called RandMark:

- When a request to a page arrives, if it is in cache, then mark it.
- If it is not in cache, then remove an unmarked page uniformly at random, fetch the requested page from disk, and mark it.
- If all pages in cache are marked when a page faults, unmark all pages first before applying the above strategy.

This algorithm is $2H_k$-competitive, where $H_k = 1 + \frac{1}{2} + \cdots + \frac{1}{k}$ is the k-th harmonic number. Here competitiveness is over expectation.

Theorem. RandMark is $2H_k$-competitive against an oblivious adversary.

Proof. We split our proof into two parts.

Part 1. The expected number of faults made by RandMark in a phase is at most cH_k, where c is the number of new pages in the phase.

Proof. Fix a phase. A page is called *new* if it did not appear in the previous phase, and *old* otherwise. Let c be the number of new pages in the current phase.

The only way an old page can fault is if it was randomly evicted earlier in the phase to make room for a new page. Since there are c new pages, there are at most c such random evictions that can hurt old pages later.

Reveal the old pages in the current phase in the order in which they are first requested. Suppose $i-1$ old pages have already been requested and marked. Then there are at least $k-i+1$ unmarked pages remaining. Since at most c of the still-unrequested old pages could have been evicted, the probability that the next old page faults is at most

$$\frac{c}{k-i+1}.$$

Summing over all possible old pages in the phase gives

$$\begin{aligned}
\mathrm{E}\left[\text{page faults in the phase}\right] &\leq \sum_{i=1}^{k} \frac{c}{k-i+1} \\
&= c\left(\frac{1}{k} + \frac{1}{k-1} + \cdots + 1\right) \\
&= cH_k.
\end{aligned}$$

□

Part 2. The amortized number of faults made by OPT during the phase is at least $\frac{c}{2}$.

Proof. Let c be the number of new pages in the current phase. By definition, these c pages were not requested in the previous phase. The previous phase contained k distinct pages, and the current phase contains c additional pages not among those k pages. Therefore, across the previous phase and the current phase, there are at least $k+c$ distinct pages.

Since OPT can hold only k pages, it must incur at least c faults across these two phases. Charging these faults equally to the two phases, the amortized number of faults made by OPT during the current phase is at least

$$\frac{c}{2}.$$

□

Thus the competitive ratio is

$$r = \max_{\sigma} \frac{\mathrm{E}\left[\textsc{RandMark}(\sigma)\right]}{OPT(\sigma)} \leq \frac{cH_k}{c/2} = 2H_k.$$

There does exist a randomized algorithm that is H_k-competitive. It is called Partition, but we will not discuss it here. However, there is one more result for this section that we will discuss.

Theorem. No randomized paging algorithm has competitive ratio better than $\Omega(\log k)$ against an oblivious adversary.

Proof. We begin our proof with an important principle. If you are familiar with Von Neumann's minimax theorem, this theorem will look familiar. If not, this will take some time to understand.

> **Yao's Principle:** To prove a lower bound against randomized algorithms, it suffices to construct a distribution over inputs such that every deterministic algorithm performs poorly in expectation on that distribution.

Equivalently, if $\mathcal{R}$ is the set of randomized algorithms, $\mathcal{A}$ is the set of deterministic algorithms, and $\mathcal{D}$ ranges over distributions on inputs, then

$$\min_{R \in \mathcal{R}} \max_{x} \mathrm{E}\left[R(x)\right] \geq \max_{\mathcal{D}} \min_{A \in \mathcal{A}} \mathrm{E}_{x \sim \mathcal{D}}\left[A(x)\right].$$

Constructing the distribution: Let S be a set of $k+1$ pages. Generate a request sequence σ of length m by choosing each request independently and uniformly at random from S.

Fix any deterministic online algorithm A. At every time step, A has only k pages in its cache, so exactly one page of S is missing from its cache. Since the

next request is uniform over S, the probability that A faults is

$$\frac{1}{k+1}.$$

Thus, over a sequence of length m,

$$\mathrm{E}\left[A(\sigma)\right] = \frac{m}{k+1}.$$

Now consider OPT. The offline optimum can serve the sequence much more efficiently. Roughly, OPT faults only after the request sequence has accumulated all $k+1$ pages since the last time it changed its cache. The expected time to see all $k+1$ pages is the coupon collector time

$$(k+1)H_{k+1}.$$

Therefore,

$$\mathrm{E}\left[OPT(\sigma)\right] = O\left(\frac{m}{(k+1)H_k}\right).$$

Hence every deterministic algorithm has expected competitive ratio at least

$$\Omega(H_k) = \Omega(\log k).$$

By Yao's Principle, no randomized online paging algorithm has competitive ratio better than $\Omega(\log k)$ against an oblivious adversary.

More Adversaries

Besides the oblivious adversary, there are two other types:

> **Definition:** The **oblivious adversary** must construct the request sequence in advance, based only on the description of the online algorithm, but before any moves are made.

Since a randomized algorithm is not totally predictable, randomization makes it hard for the oblivious adversary to generate a difficult request sequence for the algorithm.

Definition: The **adaptive online adversary** makes the next request based on the algorithm's answers to previous ones, but serves it immediately.

This "medium adversary" can be considered as an active attacker. When you make a decision, they see that decision, and can respond appropriately.

Definition: The **adaptive offline adversary** makes the next request based on the algorithm's answers to previous ones, but serves them optimally at the end.

This adversary is extremely strong. After observing the entire sequence of moves by the algorithm, the adversary then serves them optimally.

There are two important theorems[2] on the power of the last two adversaries:

Theorem. If there is a randomized algorithm that is α-competitive against any adaptive offline adversary, then there also exists an α-competitive deterministic algorithm.

Theorem. If G is a c-competitive randomized algorithm against any adaptive online adversary, and there is a randomized d-competitive algorithm against any oblivious adversary, then G is a randomized $(c \cdot d)$-competitive algorithm against any adaptive offline adversary.

These two theorems basically indicate that randomization provides no power against the adaptive offline adversary, and provides little, if any, power against the adaptive online adversary.

References and Further Reading

- IIT Delhi - Online Algorithms Notes
- Amortized Efficiency of List Update and Paging Rules (Sleator & Tarjan)

[2] Ben-David et. al, On the Power of Randomization in Online Algorithms, STOC 1990